ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ГОРОДА МОСКВЫ
«КОЛЛЕДЖ СВЯЗИ №54»
ИМЕНИ П.М. ВОСТРУХИНА

ПРАКТИЧЕСКАЯ РАБОТА № 5

МДК.09.01

Проектирование и разработка веб-приложений

Тема: «Flask и SQLite: добавление данных»

Выполнила

ст. Гр. 4-ИСП9-46

Миронова Анна Андреевна

Москва, 2026

СОДЕРЖАНИЕ

Цель работы 3

Реализованный функционал 3

Запуск 3

Средства разработки 3

Анализ сложности 3

Результаты выполнения 4

Ответы на контрольные вопросы 5

Проверка результатов 7

Заключение 8

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ И ИНТЕРНЕТ-ИСТОЧНИКОВ 9

Листинги программ 10

Цель работы

Освоить: flask и sqlite: добавление данных.

Условие: практическая работа №5.

Реализованный функционал

Добавлена POST-форма с label, required и maxlength. Сервер удаляет крайние пробелы и принимает название длиной 1–200 символов. Успешный POST сохраняет строку, фиксирует транзакцию, закрывает соединение и возвращает 302 на GET /. Ошибки ввода дают 400, ошибка БД — 503 с понятным сообщением.

Запуск

```
python web/practice_05/app.py
```

Средства разработки

Python 3.14.6, Flask 3.1.2, SQLite 3.53.3. Проверки выполнены в Arch Linux/WSL. Запуск в POCA и Python 3.8 отдельно не проверялся.

Порядок запуска: инструкция.

Анализ сложности

POST: проверка/нормализация строки O(L), вставка O(log n + L), дополнительная память O(L). Последующий GET с полным списком O(n + S) по времени и памяти. PRG добавляет сетевой переход, но не меняет асимптотику.

Результаты выполнения

Результат проверки показан на рисунке 1.

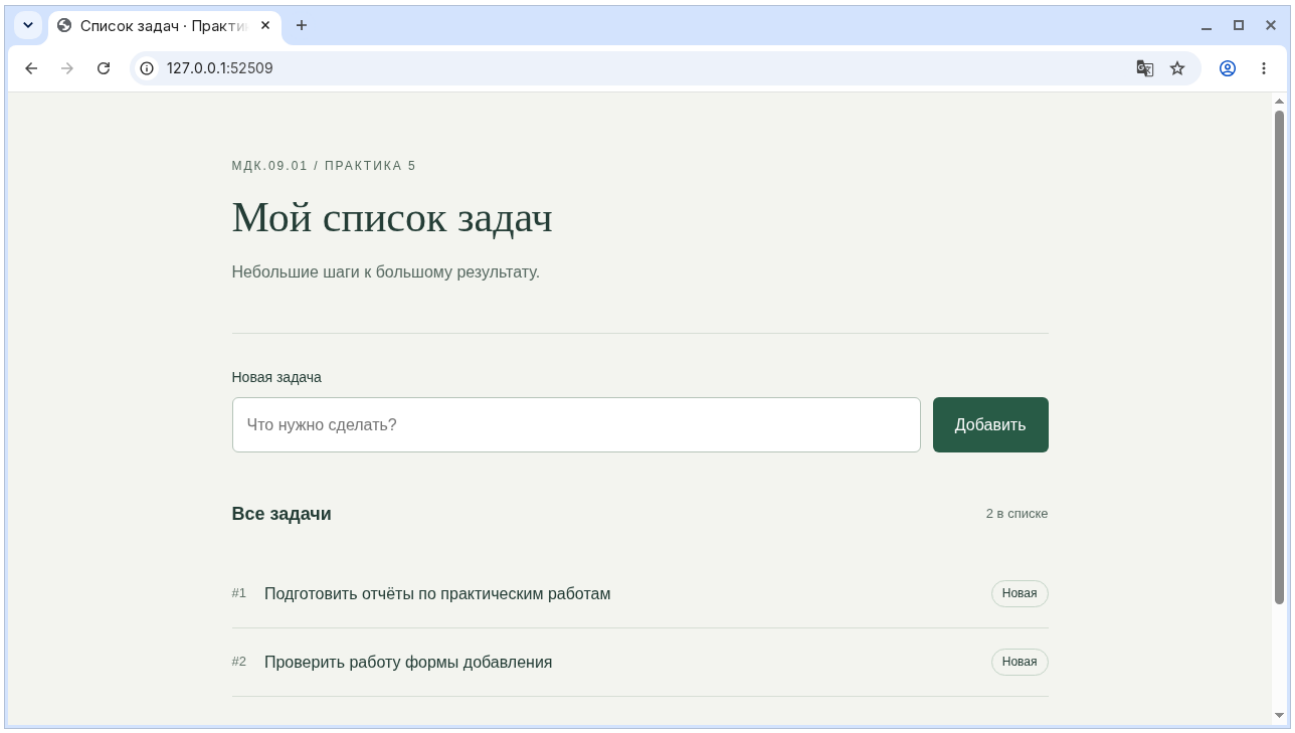


Рисунок 1 – Задачи, добавленные через форму

Результат проверки показан на рисунке 2.

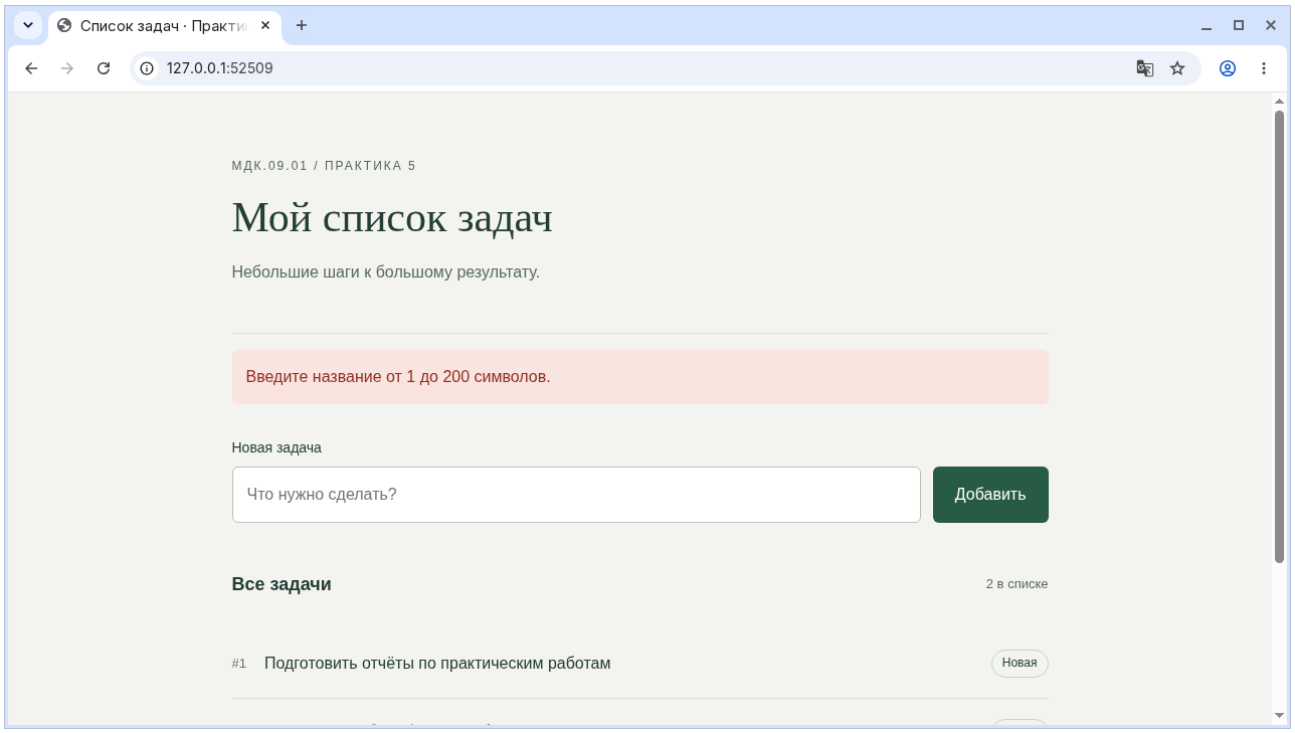


Рисунок 2 – Серверная проверка названия из пробелов

Результат проверки показан на рисунке 3.

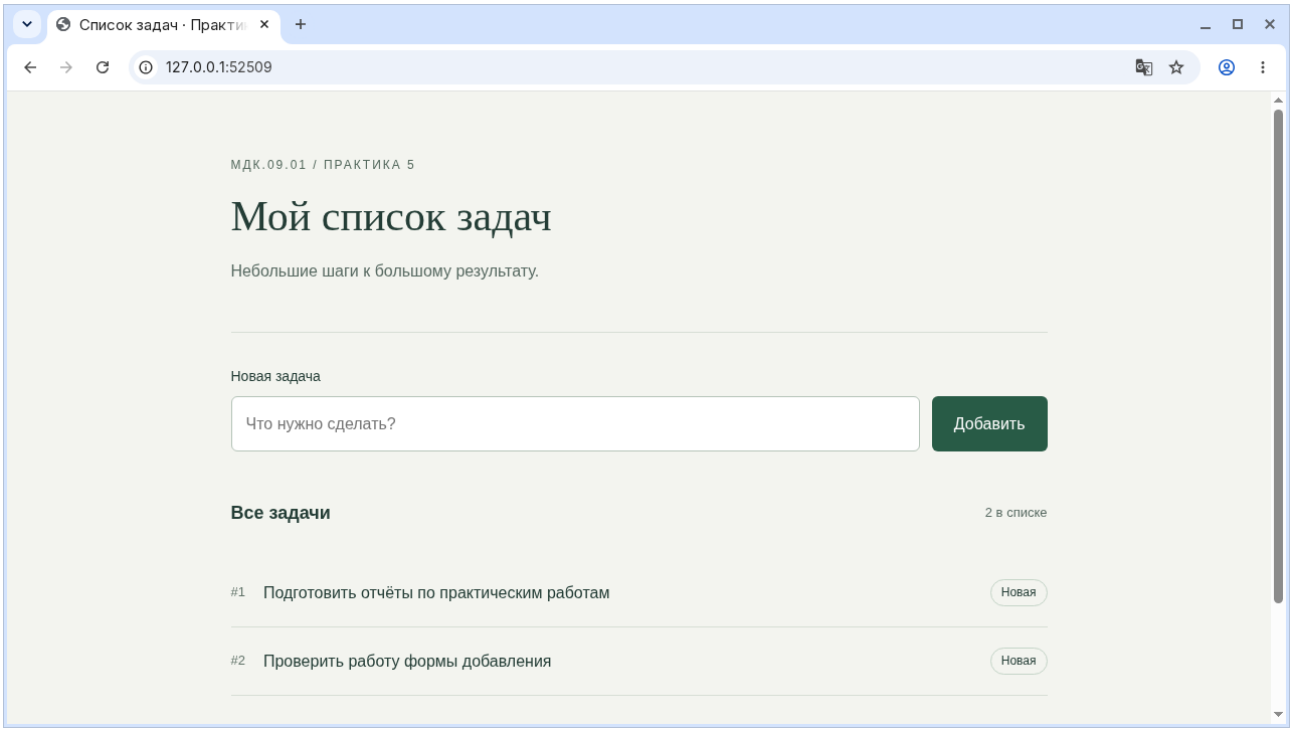


Рисунок 3 – Сохранение задач после перезапуска сервера

Ответы на контрольные вопросы

1. Почему форма отправляет POST?

Создание задачи меняет состояние. GET предназначен для безопасного чтения и может повторяться браузером, кешем или роботом.

2. Что будет без methods=[GET, POST]?

Flask оставит GET, HEAD и OPTIONS. POST получит 405 Method Not Allowed.

3. Для чего PRG?

После сохранения сервер перенаправляет на GET. Обновление итоговой страницы повторяет чтение и не создаёт вторую задачу. Без redirect браузер может предложить повторно отправить форму.

4. Почему url_for лучше строки?

Строит URL по имени обработчика с учётом префиксов и параметров; перенос маршрута не требует менять каждую ссылку.

5. Почему request.form.get безопаснее индексирования?

При отсутствии title возвращает значение по умолчанию, а не BadRequest-KeyError. Проверка пустой строки выполняется единообразно.

6. Зачем серверная валидация при required?

HTML можно изменить или обойти прямым HTTP-запросом. Пробельная строка также проходит required, поэтому сервер обязан независимо проверять содержимое.

7. Порядок транзакции?

Получить поле → strip и проверить → открыть соединение → параметризованный INSERT → commit → finally/close → redirect 302 → GET → SELECT → render_template. При SQLite-ошибке выполняются rollback и close.

Проверка результатов

Результат проверки показан на рисунке 4.

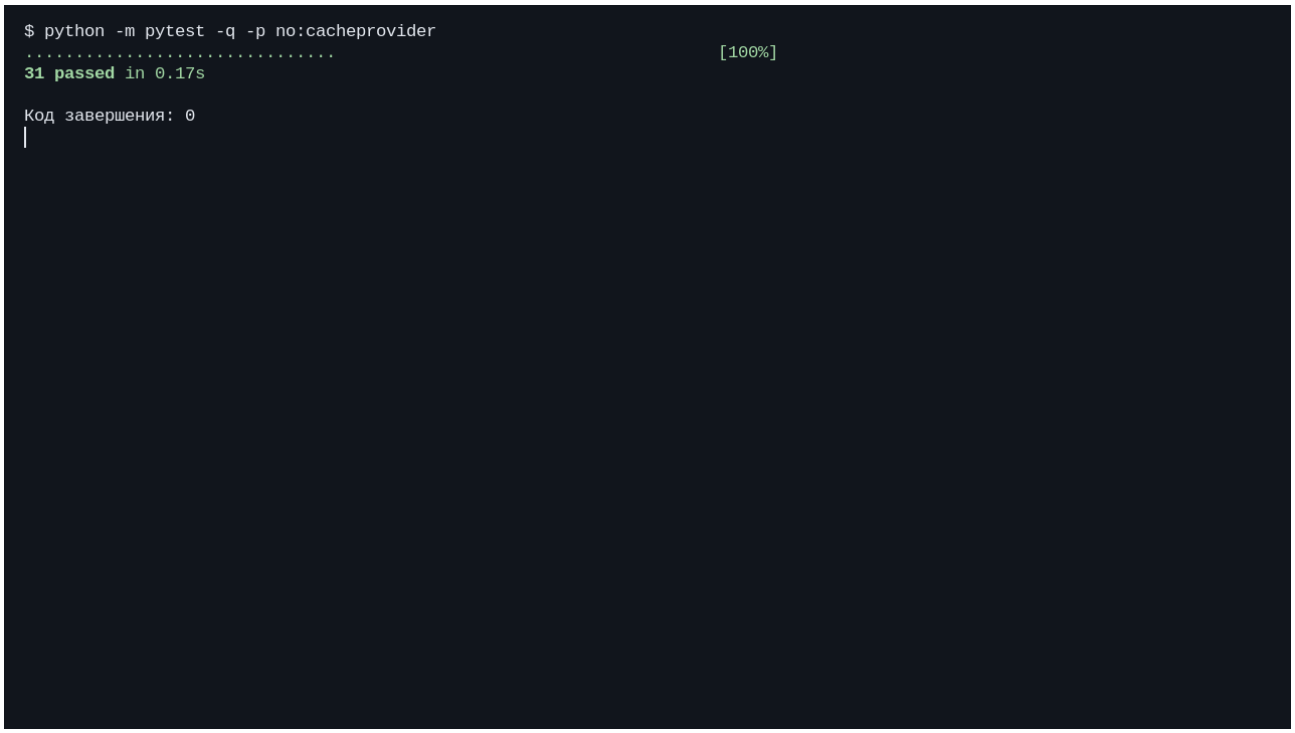


Рисунок 4 – Результат запуска автоматических Python-тестов

Заключение

Реализована форма добавления задач на той же странице, где показан список. Запрос POST проверяет название, записывает его в SQLite и возвращает перенаправление на GET. После этого новая задача отображается в списке. Перезагрузка страницы после перенаправления не отправляет форму повторно и не создаёт дубликат.

Проверены обычное добавление, пустой ввод, слишком длинное название и сохранение данных после перезапуска сервера. Отдельные автоматические тесты проверяют экранирование HTML и строку с SQL-командой: пользовательский ввод передаётся параметром запроса. При недоступной базе приложение возвращает сообщение об ошибке. В результате получено небольшое веб-приложение с постоянным хранением задач и обработкой основных ошибок ввода.

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ И ИНТЕРНЕТ-ИСТОЧНИКОВ

1. Гниденко, И.Г. Технологии и методы программирования [Текст]: учебное пособие для вузов / И.Г. Гниденко, Ф.Ф. Павлов, Д.Ю. Федоров. – 2-е изд., перераб. и доп. – Москва: Юрайт, 2024. – 248 с.
2. Жуков, Р.А. Язык программирования Python: практикум [Текст]: учебное пособие / Р.А. Жуков. – Москва: ИНФРА-М, 2026. – 216 с.
3. Федоров, Д.Ю. Программирование на Python [Текст]: учебное пособие для вузов / Д.Ю. Федоров. – 6-е изд., перераб. и доп. – Москва: Юрайт, 2025. – 187 с.
4. Фёдорова, Г.Н. Разработка, внедрение и адаптация программного обеспечения отраслевой направленности [Текст]: учебное пособие / Г.Н. Фёдорова. – Москва: КУРС: ИНФРА-М, 2024. – 336 с.
5. Чернышев, С.А. Основы программирования на Python [Текст]: учебное пособие для вузов / С.А. Чернышев. – 2-е изд., перераб. и доп. – Москва: Юрайт, 2024. – 349 с.

Электронные ресурсы

6. Документация веб-фреймворка Flask [Электронный ресурс]: https://flask.palletsprojects.com/en/stable/ (дата обращения 10.11.2026).
7. Официальное руководство по языку Python и модулю sqlite3 [Электронный ресурс]: https://docs.python.org/3/library/sqlite3.html (дата обращения 10.11.2026).
8. Справочник по реляционной СУБД SQLite [Электронный ресурс]: https://www.sqlite.org/docs.html (дата обращения 10.11.2026).

Листинги программ

Web/practice_05/app.py

```
import os
import sqlite3
from pathlib import Path
from flask import Flask, render_template, request, redirect, url_for


def create_app(db_path=None):
    app = Flask(__name__, root_path=str(Path(__file__).parent))
    app.config["DATABASE"] = str(
        db_path or os.environ.get("TASK_DB", Path(__file__).with_name("app.db"))
    )
    conn = sqlite3.connect(app.config["DATABASE"])
    try:
        conn.execute(
            "CREATE TABLE IF NOT EXISTS tasks (id INTEGER PRIMARY KEY, title TEXT NOT
NULL, status TEXT DEFAULT 'new')"
        )
        conn.commit()
    finally:
        conn.close()

    def fetch_all_tasks():
        conn = sqlite3.connect(app.config["DATABASE"])
        conn.row_factory = sqlite3.Row
        try:
            return conn.execute(
                "SELECT id,title,status FROM tasks ORDER BY id"
            ).fetchall()
        finally:
            conn.close()

    @app.route("/", methods=["GET", "POST"])
    def index():
        try:
            if request.method == "POST":
                title = request.form.get("title", "").strip()
                if not title or len(title) > 200:
                    return render_template(
                        "index.html",
                        items=fetch_all_tasks(),
                        error="Введите название от 1 до 200 символов.",
                    ), 400
                conn = sqlite3.connect(app.config["DATABASE"])
                try:
                    conn.execute("INSERT INTO tasks(title) VALUES (?)", (title,))
                    conn.commit()
                except sqlite3.Error:
                    conn.rollback()
                    raise
                finally:
                    conn.close()
                return redirect(url_for("index"))
            return render_template("index.html", items=fetch_all_tasks())
        except sqlite3.Error:
```

```
            app.logger.exception("Ошибка SQLite")
            return render_template(
                "index.html",
                items=[],
                error="Не удалось обратиться к базе данных. Повторите запрос позже.",
            ), 503

    return app


if __name__ == "__main__":
    create_app().run(
        host="127.0.0.1",
        port=int(os.environ.get("PORT", "5000")),
        debug=os.environ.get("FLASK_DEBUG") == "1",
    )
```

Web/practice_05/crud_console.py

```
import os
import sqlite3
from pathlib import Path

DB_PATH = Path(os.environ.get("TASK_DB", Path(__file__).with_name("app.db")))


def init_db(db_path=DB_PATH):
    conn = sqlite3.connect(db_path)
    try:
        conn.execute(
            "CREATE TABLE IF NOT EXISTS tasks (id INTEGER PRIMARY KEY, title TEXT NOT
NULL, status TEXT DEFAULT 'new')"
        )
        conn.commit()
    finally:
        conn.close()


def query(sql, params=(), db_path=DB_PATH, read=False):
    conn = sqlite3.connect(db_path)
    try:
        cursor = conn.execute(sql, params)
        if read:
            return cursor.fetchall()
        conn.commit()
        return cursor.lastrowid if sql.startswith("INSERT") else cursor.rowcount
    except sqlite3.Error:
        conn.rollback()
        raise
    finally:
        conn.close()


def list_tasks(db_path=DB_PATH):
    return query(
        "SELECT id,title,status FROM tasks ORDER BY id", db_path=db_path, read=True
    )


def add_task(title, db_path=DB_PATH):
    title = title.strip()
```

```
    if not title:
        raise ValueError("Название не может быть пустым")
    return query("INSERT INTO tasks(title) VALUES (?)", (title,), db_path)


def complete_task(task_id, db_path=DB_PATH):
    return query("UPDATE tasks SET status='done' WHERE id=?", (task_id,), db_path)


def delete_task(task_id, db_path=DB_PATH):
    return query("DELETE FROM tasks WHERE id=?", (task_id,), db_path)


def main():
    init_db()
    while True:
        print("\n1 — Список; 2 — Добавить; 3 — Выполнить; 4 — Удалить; 0 — Выход")
        try:
            choice = input("Команда: ").strip()
            if choice == "0":
                return
            if choice == "1":
                rows = list_tasks()
                for row in rows:
                    print(f"#{row[0]} | {row[1]} | {row[2]}")
                if not rows:
                    print("Список пуст")
            elif choice == "2":
                print("Создана задача #", add_task(input("Название: ")))
            elif choice in ("3", "4"):
                value = input("ID: ").strip()
                if not value.isascii() or not value.isdigit() or int(value) < 1:
                    print("Введите положительный целый ID")
                    continue
                result = (complete_task if choice == "3" else delete_task)(int(value))
                print("Изменено строк:", result)
            else:
                print("Неизвестная команда")
        except (sqlite3.Error, ValueError) as exc:
            print("Ошибка:", exc)
        except (EOFError, KeyboardInterrupt):
            return


if __name__ == "__main__":
    main()
```

Web/practice_05/init_db.py

```
import os
import sqlite3
from pathlib import Path

DB_PATH = Path(os.environ.get("TASK_DB", Path(__file__).with_name("app.db")))


def init_db(db_path=DB_PATH):
    conn = sqlite3.connect(db_path)
    try:
        conn.execute(
            "CREATE TABLE IF NOT EXISTS tasks (id INTEGER PRIMARY KEY, title TEXT NOT
```

```
NULL, status TEXT DEFAULT 'new')"
        )
        conn.commit()
    finally:
        conn.close()


if __name__ == "__main__":
    init_db()
    print("База данных и таблица tasks успешно созданы.")
    print(f"Путь: {DB_PATH.resolve()}")
```

Web/practice_05/static/style.css

```
*{box-sizing:border-box}body{margin:0;background:#f3f4ef;color:#243d36;font:16px/1.6
Arial,sans-serif}main{max-width:860px;margin:64px auto;padding:0 24px}header{border-
bottom:1px solid #d8dfd6;padding-bottom:32px}.eyebrow{font-size:12px;letter-spac-
ing:2px;color:#53685c}h1{font:normal 42px/1.2 Georgia,serif;margin:16px 0}.intro{col-
or:#586a62}form{margin:32px 0}label{display:block;font-size:14px;margin-bot-
tom:8px}.form-row{display:flex;gap:12px}input{min-width:0;flex:1;border:1px solid
#b6c5ba;border-radius:6px;padding:14px;font:inherit;background:#fff}button{bor-
der:0;border-radius:6px;background:#285b46;color:white;padding:12px 22px;font:inher-
it;cursor:pointer}button:hover{background:#19432f}input:focus-visible,button:focus-vis-
ible{outline:3px solid #94b9a3;outline-offset:3px}section{margin-top:32px}.section-
head{display:flex;justify-content:space-between;align-items:center}.section-head
span{font-size:13px;color:#586a62}h2{font-size:18px}ul{list-style:none;padding:0}
li{display:flex;gap:18px;align-items:center;padding:20px 0;border-bottom:1px solid
#d8dfd6}.number{font-size:13px;color:#617369}.title{flex:1;overflow-wrap:anywhere;min-
width:0}.status{font-size:12px;padding:3px 10px;border:1px solid #c6d6c9;border-radi-
us:20px;white-space:nowrap}.done .title{text-decoration:line-through;col-
or:#65756d}.done .status{background:#e3eddf}.empty{padding:34px 24px;back-
ground:#e9ede5;border-radius:8px;color:#50645a}.error{background:#f9e4df;col-
or:#922f25;padding:14px;border-radius:6px}footer{display:flex;justify-content:space-be-
tween;gap:16px;border-top:1px solid #d8dfd6;margin-top:56px;padding-top:18px;font-
size:12px;color:#5d7065}@media(max-width:540px){main{margin:32px auto;padding:0 18px}
h1{font-size:32px}.form-row{flex-direction:column}li{gap:10px}.status{font-size:11px}
footer{flex-direction:column;gap:4px}}
```

Web/practice_05/templates/index.html

```
<!doctype html>
<html lang="ru">
<head><meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-
scale=1"><title>Список задач · Практика 5</title>
<link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}"></head>
<body><main><header><p class="eyebrow">МДК.09.01 / ПРАКТИКА 5</p><h1>Мой список
задач</h1><p class="intro">Небольшие шаги к большому результату.</p></header>
{% if error %}<p class="error" role="alert">{{ error }}</p>{% endif %}
<form method="post"><label for="title">Новая задача</label><div class="form-row"><input
id="title" name="title" placeholder="Что нужно сделать?" required maxlength="200" auto-
complete="off"><button type="submit">Добавить</button></div></form>
<section aria-label="Задачи"><div class="section-head"><h2>Все
задачи</h2><span>{{ items|length }} в списке</span></div>
{% if items %}<ul>{% for item in items %}<li class="{{ 'done' if item.status == 'done'
else '' }}"><span class="number">#{{ item.id }}</span><span class="title">{{ item.ti-
tle }}</span><span class="status">{{ 'Выполнено' if item.status == 'done' else
'Новая' }}</span></li>{% endfor %}</ul>
{% else %}<div class="empty"><h3>Список пуст</h3><p>Добавьте первую задачу с помощью
формы.</p></div>{% endif %}</section>
```

```
<footer>Python · Flask · SQLite <span>Учебный проект / 2026</span></foot-
er></main></body></html>
```